Now you can define an empty Task class that inherits from ActiveRecord::Base, and you can use objects of that class to interact with the database:

```
class Task < ActiveRecord::Base; end

task = Task.new
task.description = 'Clean up garage'
task.completed = true
task.save

task.description    # => "Clean up garage"
task.completed?     # => true
```

The previous code calls four accessor methods to read and write the object's attributes: two write accessors (description= and completed=), one read accessor (description), and one query accessor (completed?). None of these *Mimic Methods (218)* comes from the definition of Task. Instead, Active Record generated them by looking at the columns of the tasks table. These automatically generated accessors are called *attribute methods*.

You probably expect that attribute methods such as description= are either *Ghost Methods (57)* implemented through method_missing or *Dynamic Methods (51)* defined with define_method. Things are actually more complicated than that, as you'll find out soon.

A History of Complexity

Instead of looking at the current implementation of attribute methods, let me go all the way back to 2004—the year that Rails 1.0.0 was unleashed on an unsuspecting world.

Rails 1: Simple Beginnings

In the very first version of Rails, the implementation of attribute methods was just a few lines of code:

gems/activerecord-1.0.0/lib/active_record/base.rb

```
module ActiveRecord
  class Base
    def initialize(attributes = nil)
      @attributes = attributes_from_column_definition
      # ...
    end

    def attribute_names
      @attributes.keys.sort
    end
```